AARTHIK · FINTECH PRODUCT DESIGN

Money rarely arrives on schedule.

A clearer answer to “What can I afford today?” for people with irregular income.

Sahil Sahu / Product design & strategy

The supplied study originally used the working name Meridian. The original research and design account is followed by a review of the current Aarthik demo. Its visible navigation and sample-data flows have evolved from the design specification; bank connectivity, money movement and paid usage are not established by the demo.

Explore the prototype

Inside this study

Executive Summary

The Problem

Research & Discovery

Problem Framing

Ideation

Prioritization

Structuring the Product

Visual & Interaction Design

Prototyping & Usability Testing

System Architecture & Tech Stack

The current app: what a visitor can see

Monetisation: charge for useful planning, not financial anxiety

How Aarthik could become a sustainable business

What must be true before this model is sold

Success Metrics

What This Case Study Won't Pretend

AARTHIK

A finance app for people whose income refuses to behave

A COMPLETE PRODUCT DESIGN CASE STUDY

Research · frameworks · ideation · prioritization · testing · architecture

Executive Summary

I get paid four different ways, on four different schedules, in four different amounts — and every finance app on my phone was built for someone who gets one number, on the same date, every month.

A fixed monthly budget did not reflect the irregular-income situations this project set out to explore.

Aarthik is a mobile finance product designed around three refusals: it refuses to assume one income, it refuses to reduce money to raw numbers, and it refuses to treat financial literacy as a bolt-on feature nobody opens twice. This document walks through the entire process behind it — every research method, framework, and design decision used to get from a personal irritation to a fully specified, architected product.

Stage	What happened
Research	8 qualitative interviews, competitive audit of 6 apps, empathy mapping, 2 personas, JTBD synthesis
Framing	Affinity-mapped insights, 3 How Might We statements, one sharpened problem statement
Ideation	Crazy 8s sprint, brainwriting, SCAMPER pass, MoSCoW / RICE / Kano prioritization
Design	30+ screens, one unified dark premium design system, user journey mapping
Validation	Think-aloud usability sessions, five-second test, System Usability Scale (SUS) scoring
Engineering	Full system architecture and tech stack specified for build-readiness

The Problem

The starting concern was an onboarding flow that asks for a single monthly income. That assumption does not describe every earner, and it became the constraint I wanted this design to address.

For the irregular-income users considered in this study, that assumption is a poor fit. Freelancers invoice on their own timeline. Gig workers get paid per job. Side-hustlers get paid whenever a client feels like it. Even people with a conventional job increasingly stack two or three income streams on top of it.

The apps aren't broken. They're just answering a question nobody with irregular income actually asked.

The real cost isn't inconvenience — it's anxiety. A good week feels like free money, so it gets spent. A lean week two weeks later turns into panic, because no system ever marked the difference between a real surplus and a temporary illusion of one.

Research & Discovery

Secondary research

Before talking to anyone, I audited six major finance and budgeting apps for a week each, logging what job each one was doing, where it broke down, and what assumptions it silently made about its users. Almost all of them assumed steady, single-source income and financial literacy the user may not have had. I also read through hundreds of one- and two-star app store reviews — reliably the richest, most unfiltered source of pain points available for free.

Primary research — qualitative interviews

I interviewed 8 people across three segments: full-time freelancers, gig-platform workers, and salaried employees running a side hustle. Sessions followed a semi-structured discussion guide, asked about specific past behavior rather than hypotheticals (“walk me through last week”, not “would you use”), and were recorded with consent for later synthesis using the think-aloud style of narration.

Empathy map

Interview notes were synthesized into an empathy map to separate what people said from what they actually did and felt — the gaps between those quadrants are usually where the real insight lives.

SAYS “I'm pretty good with money, I just don't always know what's coming in.” “I have a system, it's just in my head.”	**THINKS If I don't spend this now, will I actually remember I ‘saved’ it, or will it just vanish into my balance?**
DOES Keeps a running mental buffer. Checks balance obsessively after invoicing a client. Avoids linking all accounts to one app out of habit, not distrust.	FEELS Quietly anxious between paydays. Guilty after spending during a good week, even when the spend was reasonable.

Personas

Two personas anchored every design decision that followed — not as decorative artifacts, but as a running test: would this actually help Priya, or would it help Arjun instead?

Priya Nair, 29 Freelance graphic designer · Mumbai · 3 active clients Priya invoices on her own schedule and gets paid anywhere from 15 to 60 days after sending a bill. Some months bring in triple her old full-time salary; others bring in almost nothing. “I've genuinely lost track of whether I'm ahead or behind for the month until I open a spreadsheet and just start typing numbers in.” Goals Know instantly how much of today's balance is actually safe to spend Stop overspending in a good month at the cost of a lean one two months later Frustrations Every app asks for a fixed monthly income during onboarding and never lets her correct it later No tool distinguishes a real surplus from a temporary illusion of one

Arjun Mehta, 24 Gig delivery partner · Bengaluru · also tutors on weekends Arjun earns from two gig platforms plus weekend tutoring. Daily payouts vary with demand, weather, and platform incentives — he's never sure what a ‘normal’ week even looks like. “I know the vocabulary — credit score, interest — but if you asked me to actually explain how either works on my own card, I couldn't.” Goals See all his income sources in one place without manually adding each payout Learn how credit and interest actually work, tied to his own numbers, not a textbook example Frustrations Financial literacy content he's found online is generic and forgettable No single dashboard reflects the true shape of a multi-source income week

Jobs to be done

- When I get paid from an unpredictable client, I want to know instantly how much is safe to spend, so I can enjoy a good week without paying for it during the next lean one.
- When I have income landing from more than one source, I want one place that reflects all of it, so I stop reconstructing my own finances in a spreadsheet.
- When I don't fully understand a financial concept that affects me, I want to learn it through my own numbers, so it actually sticks instead of feeling like homework.

Affinity-mapped key insights

- Nobody wanted a monthly budget — everyone wanted an answer to one question, right now: can I spend this today?
- Good weeks were described as dangerous, not lucky — extra income got spent precisely because nothing marked it as a buffer instead of available balance.
- Financial literacy wasn't a gap people admitted to, but nearly everyone's follow-up answers revealed they knew vocabulary, not mechanics.

Problem Framing

How Might We

How might we help people with irregular income know, at any moment, whether their current balance is safe to spend or just an illusion of surplus?

How might we make multiple income sources feel like one coherent financial picture instead of several disconnected apps and mental math?

How might we teach real financial concepts in a way that feels earned and personal, rather than generic and skippable?

Point of view / problem statement

People earning irregular, multi-source income can't tell the difference between a safe-to-spend surplus and a dangerous illusion of one — and need a clearer way to see that distinction.

Ideation

Crazy 8s

The core interaction model — how to visually communicate “safe to spend” — came out of a Crazy 8s sprint: 8 distinct sketches in 8 minutes, forcing quantity over polish so the obvious, boring first idea (a plain balance number) couldn't win by default. Concepts ranged from a traffic-light indicator to a weather-style forecast metaphor to a single confident numeral with a color-coded ring — the numeral-plus-ring concept is what eventually became the home screen's central figure.

Brainwriting & SCAMPER

For the financial literacy hub specifically, I ran a brainwriting round (6-3-5 style) to generate lesson-delivery formats without one voice dominating the direction, then applied SCAMPER to an existing competitor's course-list format — substituting a linear course list for a game-style progression path, and combining quiz content with the user's own transaction data instead of generic examples.

Concept selection

Every concept was scored against the problem statement, not against novelty for its own sake: did it help someone tell a real surplus from a fake one, see all their income in one place, or learn something true about their own money? Concepts that were merely clever but didn't answer one of the three HMWs were cut regardless of how well they sketched.

Prioritization

MoSCoW

Must have

- Multi-source income tracking, synced in real time
- Safe-to-spend indicator on the home screen
- Secure account linking via an account aggregator

Should have

- Behavioral spend/earn insights and smart overspend alerts
- Financial health score

Could have

- Gamified learning hub with streaks and badges
- Goals and savings tracker, voucher marketplace

Won't have (v1)

- Shared / couple finance mode
- Investment portfolio tracking, multi-currency support

RICE scoring

Reach and Impact scored out of 10, Confidence as a percentage, Effort in person-weeks — RICE score = (Reach × Impact × Confidence) ÷ Effort. Higher scores informed the proposed build order.

Feature	Reach	Impact	Confidence	Effort	RICE
Multi-income tracking	9	9	90%	5	14.6

Feature	Reach	Impact	Confidence	Effort	RICE
Irregular-income budget planner	8	9	85%	6	10.2
Behavioral nudges & alerts	8	8	80%	6	8.5
Gamified literacy hub	6	6	70%	7	3.6
Shared / couple finance mode	5	5	60%	8	1.9

Kano model

Category	Features
Basic (expected)	Biometric login, transaction history, secure account linking
Performance (more = better)	Speed and granularity of income tracking, accuracy of behavioral insights
Delighter	Safe-to-spend-today figure, spending-personality card, transaction-based learning quizzes

Structuring the Product

User journey map

Stage	Action	Thought	Emotion	Opportunity
Discovery	Finds Aarthik after a frustrating month with a fixed-income app	“Maybe this one actually gets it”	Cautiously hopeful	Onboarding must ask about income shape, not one number
Onboarding	Links multiple income sources one by one	“It's actually letting me add all of them”	Relief	Celebrate each linked source as progress, not a form field
Payday spike	A large freelance payment lands	“Is this actually mine to spend?”	Excited, then unsure	Safe-to-spend figure must update instantly and explain itself
Lean week	Income is low, spending continues out of habit	“Am I about to overspend?”	Anxious	A proactive nudge before the overspend, not a report after

Stage	Action	Thought	Emotion	Opportunity
Learning moment	Gets a nudge tied to a real transaction	“Oh — I didn't actually know that”	Curious, mildly proud	Tie every lesson to the user's own numbers

Information architecture

Five primary navigation destinations sit behind the bottom tab bar — Home, Insights, Add (a raised central action), Learn, and Profile — with every other screen reachable within two taps from one of these five. Account linking, security, and notification settings live one level deeper under Profile so they stay out of daily traffic without being hard to find.

Visual & Interaction Design

Design system

The visual direction leans deliberately into a dark, premium, high-contrast aesthetic, closer to a private banking app than a consumer budgeting tool — a near-black base (#0A0A0A–#121212), a single vivid lime-yellow accent (#E8FF3D) reserved exclusively for the user's own money and primary actions, generous rounded corners, and a bold geometric sans-serif typeface throughout.

Screen inventory

Thirty-plus screens were designed as one continuous system — identical corner radii, spacing, and type scale throughout — grouped into six flows:

- Onboarding & authentication — splash, carousel, sign-up, OTP, biometric setup, account linking (6 screens)
- Home & income dashboard — swipeable multi-income cards, net worth summary, income source detail (2 screens)
- Behavioral insights — spend/earn analytics, spending breakdown, cash-flow calendar, smart alerts, irregular-income planner, transactions (7 screens)
- Financial literacy hub — progression-path home, lesson cards, quizzes tied to real transactions, badge unlocks, financial health score (5 screens)
- Goals & payments — goals list and detail, top-up, send money, vouchers, card management (7 screens)
- Profile & settings — profile, notifications, security (3 screens)

Prototyping & Usability Testing

Think-aloud protocol

Interactive prototypes were tested with 5 participants per round, narrating their thoughts in real time while completing specific tasks (“set up a budget for a week where you expect variable income”, “find out if you can afford a ₹2,000 purchase right now”). This surfaced confusion — the first version of the safe-to-spend figure was read by 3 of 5 participants as their entire balance, not a recommendation — in real time rather than after the fact.

Five-second test

The home screen was shown to 8 new participants for five seconds, then hidden, and each was asked what they remembered. Every participant recalled the large safe-to-spend numeral; only 2 of 8 recalled the net worth figure beneath it — confirming the visual hierarchy was correctly weighted toward the number that actually mattered.

System Usability Scale (SUS)

After the second testing round, participants completed the standard 10-item SUS questionnaire. One representative response set, scored using the standard SUS formula:

Item (odd = positive, even = negative)	Response (1–5)	Contribution
1. Would like to use this frequently	4	3
2. Found it unnecessarily complex	2	3
3. Thought it was easy to use	5	4
4. Would need support to use it	2	3
5. Found functions well integrated	4	3
6. Found too much inconsistency	2	3
7. Would imagine people learning fast	5	4
8. Found it very cumbersome	1	4
9. Felt very confident using it	4	3
10. Needed to learn a lot first	2	3

Total contribution 33 × 2.5 = 82.5 — a Good score, comfortably above the industry-average benchmark of 68, with the biggest remaining friction point (item 4) flagged directly back into the next design iteration.

Iteration

The clearest before/after: the original safe-to-spend card showed a number with no context. After testing revealed participants couldn't tell if it was good or bad news, the revised card added a single-word state label above the number (“Comfortable”, “Tight”, “Overspending”) and a color-coded ring — the same visual language validated back in the Crazy 8s sprint.

System Architecture & Tech Stack

The following is the proposed architecture from the design phase, not a description verified against a deployed app. It groups responsibilities into client, gateway, services, data and external integrations. Provider capabilities, access arrangements and the final implementation still need to be checked.

Layer	Responsibility
Client apps	Flutter mobile app and web dashboard — the only layer the user directly touches
API gateway	Authentication, rate limiting, and request routing to the correct backend service
Core microservices	Transactions, income tracking, behavioral insights, notifications, financial literacy — each an independently deployable service
Data layer	PostgreSQL for transactional data, TimescaleDB for time-series trends, Redis for cache and live balances
External integrations	Account aggregator for bank/income linking, payment gateway for transfers and top-ups

Tech stack

Layer	Choice & why
Mobile app	Flutter (Dart) — one codebase for iOS and Android with the smooth, high-frame-rate animation a premium finance UI needs
Backend API	Node.js with NestJS (TypeScript) — strongly typed, fast to iterate, a natural fit for a microservices API layer

Layer	Choice & why
Core database	PostgreSQL, extended with TimescaleDB for time-series income and spending data
Cache & real-time layer	Redis — live balance updates and session state without hammering the primary database
Behavioral insights engine	Python microservice (FastAPI) combining rule-based categorization with lightweight ML (scikit-learn)
Account linking	Account Aggregator providers such as Setu / Finvu considered for consent-based access; no live connection or provider relationship is established here
Payments	Razorpay considered as a payment provider; individual use cases, eligibility and capabilities require confirmation
Auth & security	Proposed session authentication with Firebase Auth; device biometric access needs a separate platform implementation
Notifications	Firebase Cloud Messaging, backed by a custom in-app nudge engine
Infrastructure	Proposed Dockerized services behind an API gateway on AWS; deployment and scaling are not verified

The current app: what a visitor can see

The public Aarthik app is now available. After a browser-local email entry and an introduction, the demo opens an overview with an available balance, a daily safe-to-spend amount, subscriptions, transactions and cards. It offers Overview, Earnings, Assistant, Payments and Insights in its current navigation. These screens have evolved from the earlier Home, Insights, Add, Learn and Profile structure documented above.

The overview makes the variable-income direction visible with “Every way you earn” and “Track daily wages, gigs & projects.” The inspected Insights view includes cash flow, spending and budget entry points, with sample behavioural summaries. The displayed balances and insights are demo content, not evidence of connected bank accounts, executed payments or verified personalised financial analysis. The original architecture and research account above describes the design process; it should not be read as a release checklist for the current demo.

Monetisation: charge for useful planning, not financial anxiety

The model I would test for Aarthik is a consumer subscription. The product's design centres on irregular income, a daily spending estimate and planning around uneven payments. A subscription would need to earn its place through repeated usefulness, not through the promise of guaranteed financial improvement.

This is a commercial proposal. The public app demonstrates the product experience; the inspected screens do not establish a live paywall, payment collection, bank connectivity or paying customers. I have not assigned a launch price or forecast revenue from screens alone.

I would first test whether people return to the basic income-and-obligations view and understand how its spending estimate is calculated. A possible free tier would support a useful basic view. A possible paid tier could add longer planning horizons, scenario comparisons and richer history, if those capabilities are built and people demonstrate a need for them. These tier boundaries are proposals, not an inventory of existing app features.

The explanation behind a spending estimate should remain accessible. Someone should not have to pay to discover which bills or income assumptions produced a number they are being asked to trust. Missing data and uncertain future income should be visible, regardless of tier.

I would defer lending referrals, investment commissions and a voucher marketplace as launch revenue sources. They are not established commercial arrangements in this project. They could also pull the experience away from helping someone judge a purchase toward encouraging another financial transaction.

How Aarthik could become a sustainable business

The immediate user value is less work reconstructing a financial picture and a clearer distinction between money received and money available after commitments. For a freelancer, that could support day-to-day cash planning. It does not make Aarthik an accounting, tax-filing, invoicing or payroll product.

The business hypothesis is that a useful recurring planning task can support a recurring subscription. I would measure whether users return to the planning view through both high-income and low-income periods, whether the inputs stay current, and whether people can explain the output correctly. Opening the app frequently is not enough if it leaves someone more confused.

A paid pilot would test willingness to pay after people have used the relevant features. I would track trial-to-paid conversion, renewal, cancellations and refunds alongside complaints about incorrect or misunderstood estimates. An attractive price shown in a prototype is not payment evidence.

At the operating level, contribution per paid subscriber would be subscription revenue excluding taxes and refunds, less payment processing, data access, hosting and attributable support costs. The cost of supporting free users also has to be covered by the business. Bank-data or partner fees, if integrations are introduced, belong in that calculation rather than being treated as free infrastructure.

Acquisition payback would depend on observed contribution and renewal, not an assumed lifetime value. I would not publish a payback period before there is enough paid usage to estimate it. A simpler product that people trust and renew may be more viable than a broad feature set with costly integrations.

What must be true before this model is sold

The account-aggregator, payment, authentication and infrastructure choices in the original study are proposed architecture. They are not verified integrations in the available app evidence. Provider names identify design options, not partnerships or proof that every listed operation is supported.

Likewise, the research and prototype-testing figures earlier in this study come from the supplied design document. The representative SUS response is not a production satisfaction score. Neither those figures nor this commercial model establish revenue, reduced financial stress or improved saving behaviour.

Before charging for an automated spending estimate, I would validate the calculation against known inputs, show how stale or missing data affects it, and make it possible to correct assumptions. Before charging for a planning feature, I would verify that the feature is present and dependable in the paid version. The accurate description today is an interactive finance demo with a proposed subscription model; a live commercial service still needs validation.

Success Metrics

The design document proposes the following measures for future validation; they are not current production results:

- Financial health score — a custom 0–100 composite of saving habits, spending control, income stability, and debt management, recalculated weekly
- Net Promoter Score (NPS) — tracked quarterly post-launch as a loyalty signal
- Customer Satisfaction (CSAT) — captured immediately after each smart alert or nudge interaction
- Activation — percentage of new users linking a second income source within 7 days

- Retention — percentage of users still checking the safe-to-spend figure weekly after 90 days

What This Case Study Won't Pretend

This study records the original design and prototype work alongside a review of the current public Aarthik demo. A production user base, paid usage and live financial integrations are not established by that demo.

- The behavioral insight engine is designed and specified; its accuracy is a hypothesis to validate against real transaction data, not a measured result
- Usability testing so far covers prototype walkthroughs with 5–8 participants per round, not a live cohort — the next step is a closed beta with the exact irregular-income users interviewed during research
- Shared/couple finance mode was deprioritized in the design exercise; demand and priority should be tested before a later release

The measure of Aarthik isn't whether it looks like a premium finance app. It's whether it can finally answer the one question every irregular earner keeps asking their bank account and never quite getting a straight answer to: can I actually afford this, today, right now.

End of field note / Sahil Sahu